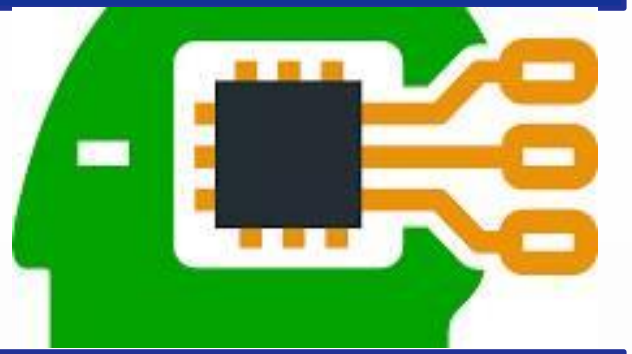
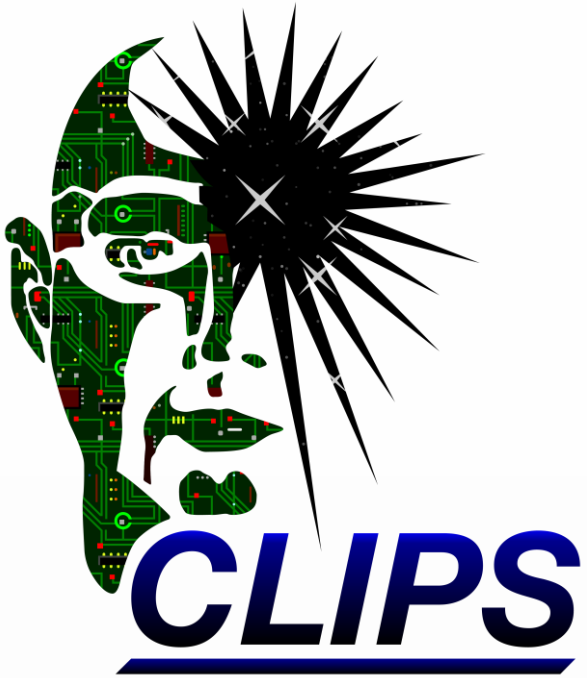
Expert Systems in CLIPS



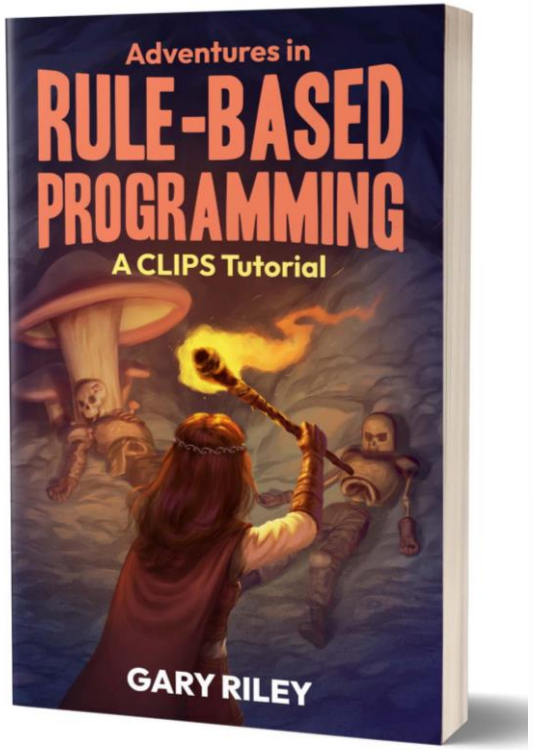
VUB
ARTIFICIAL
INTELLIGENCE
RESEARCH GROUP

CLIPS – A tool for building Expert Systems



- Developed at NASA's Johnson Space Center from 1985 to 1996, the C Language Integrated Production System (CLIPS) is a rule-based programming language useful for creating expert systems and other programs where a heuristic solution is easier to implement and maintain than an algorithmic solution.
- Written in C for portability, CLIPS can be installed and used on a wide variety of platforms. Since 1996, CLIPS has been available as public domain software.

Adventures in Rule-based Programming



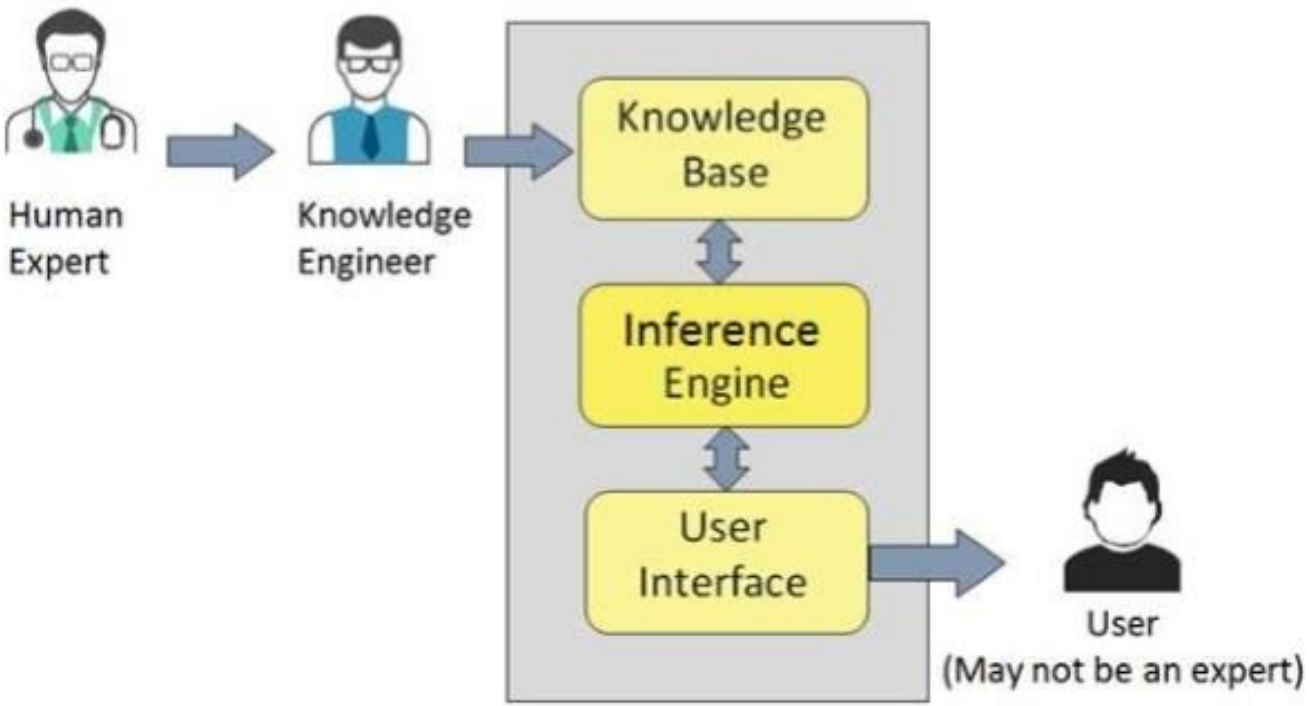
Published in 2022, this game-based tutorial covers the basics of developing applications in CLIPS. Copies can be purchased in paperback, hardback, and eBook formats.

Documentation (Available in PDF):

- Basic and Advanced Programming Guides
- Interfaces Guide

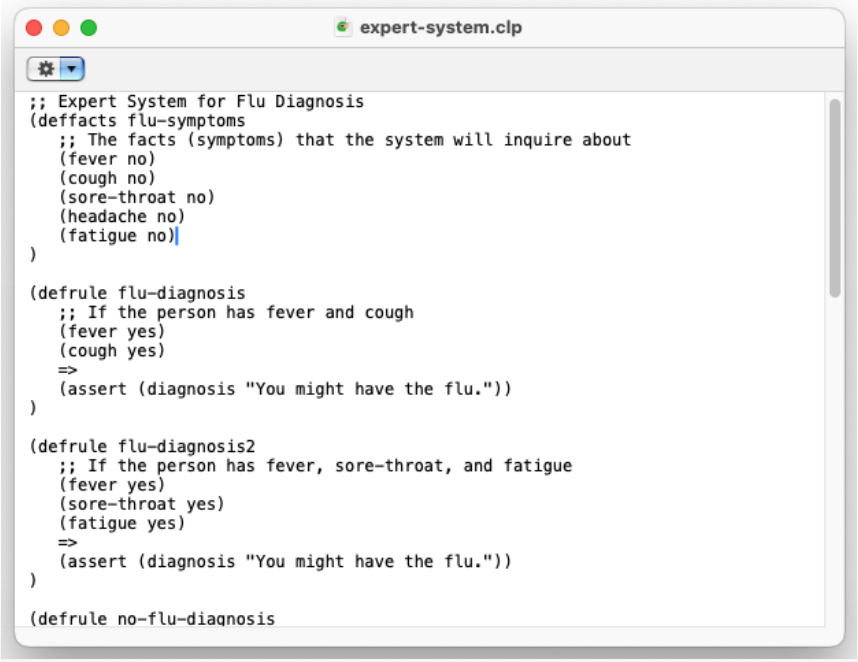
How to build an Expert System with CLIPS

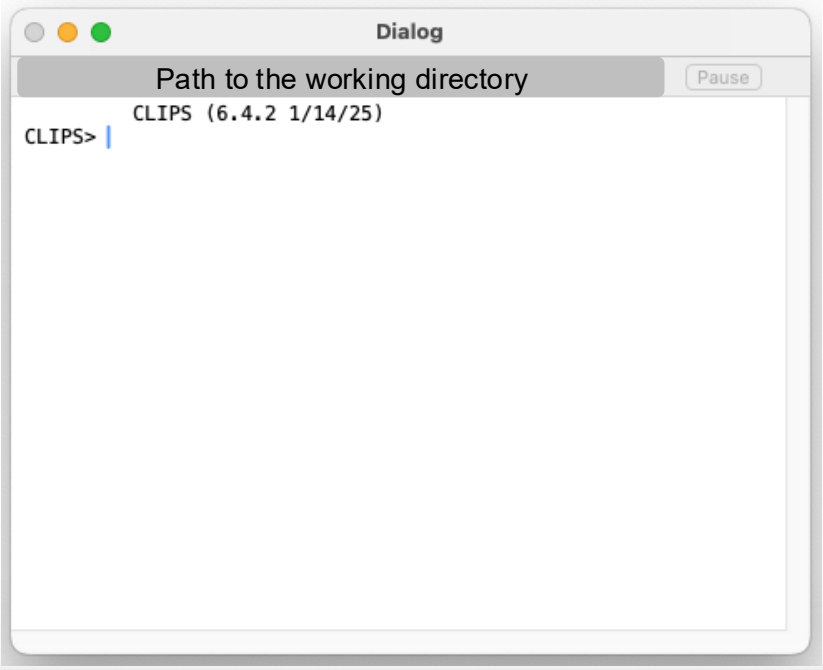
- First, an expert creates a knowledge base which is basically a set of facts related to specific domains.
- Then a set of rules is designed based on this knowledge base and different queries that the user is going to answer.
- Finally, an inference engine binds the knowledge base and the set of rules for the user to interact with, via a neat user interface.



How to build an Expert System with CLIPS

- The CLIPS IDE can be downloaded from -> http://clipsrules.sourceforge.net/
- A CLIPS program has a .clp extension (it consists of rules, facts and objects)





How to build an Expert System with CLIPS

- The inference engine decides which rules should be executed and when.
- A rule-based expert system written in CLIPS is a data-driven program where the facts, and objects if desired, are the data that stimulate execution via the inference engine.
- To begin CLIPS, just enter the appropriate run command for your system. You should see the CLIPS prompt appear as follows:

```
CLIPS>
```

- At this point you can start entering commands directly into CLIPS

Basic CLIPS commands

- (exit): to exit from CLIPS
- (clear): to clear the environment from facts, rules and other active definitions
- (reset): to set the fact base to its initial state. Recommended before each program run!
- (load “filename.clp”): loads a CLIPS program into the interpreter
- (run): executes the currently loaded program

Basic CLIPS commands

- (facts): to display a list of currently active facts in the facts database
- (rules): to display a set of rules currently in the rule base
- (agenda): to display all potential matches of active facts for all rules
- (deffacts <name> <fact1> <fact2>…): to define a set of facts
- ; indicates a line comment

Defining Facts

- Facts can be asserted:
  CLIPS > (assert (today is sunday))
  <Fact-0>

- Facts can be listed
  CLIPS > (facts)
  f-0 (today is sunday)

- Facts can be retracted
  CLIPS > (retract 0)
  CLIPS > (facts)

A set of facts:

```
(deffacts animal-fact-base
(animal dog)
(animal cat)
(animal duck))

(deftemplate father-of
        (slot father)
        (slot child) )
(deftemplate parents-of
        (slot mother)
        (slot father)
        (slot child) )
```

```
(assert (father-of (father "John") (child "Alice")))
(assert (father-of (father "Mike") (child "Bob")))
```

Defining Rules

- Rules have the following structure

```
(defrule <rule-name>
   "Optional comment"
   <conditions>
   =>
   <actions>
)
```

```
(defrule find-cat
   "Find out if there is a cat"
   (animal cat)
   =>
   (printout t "cat found" crlf)
)
```

- t indicates print to console
- crlf adds a new line after the text

Defining Variables

- The name of a variable, or variable identifier, is always written as:

  ?<variable-name>

- The ? symbol is a wildcard, it matches any value stored in exactly one field in the pattern entity.

Example

animals.clp

```
(deffacts startup  ; To assert 3 facts every time the system is reset
  (animal dog)
  (animal duck)
  (animal cat)) ; even if they are retracted they will reappear after a (reset)

(defrule animal   ; prints "animal found" 3 times
  (animal ?)
  =>
  (printout t "animal found" crlf))

(defrule list-animals   ; prints the name of each animal found
(animal ?name)
=>
(printout t ?name " found" crlf))
```

? Is a wildcard

the rule will fire 3 times

If you don't use *deffacts*, you have to assert the facts in the console or dialog before executing the program

To run a program:

(clear)

(load "filename.clp")

(reset)

(run)

CLIPS does not have an explanation module integrated

Example

animals.clp

```
(deffacts startup  ; To assert 3 facts every time the system is reset
  (animal dog)
  (animal duck)
  (animal cat)) ; even if they are retracted they will reappear after a (reset)

(defrule animal   ; prints "animal found" 3 times
  (animal ?)
  =>
  (printout t "animal found" crlf))

(defrule list-animals   ; prints the name of each animal found
(animal ?name)
=>
(printout t ?name " found" crlf))
```

```
CLIPS> (clear)
CLIPS> (load "animals.clp")
$**
TRUE
CLIPS> (reset)
==> f-1     (animal dog)
==> f-2     (animal duck)
==> f-3     (animal cat)
```

```
CLIPS> (agenda)
0      animal: f-3
0      list-animals: f-3
0      animal: f-2
0      list-animals: f-2
0      animal: f-1
0      list-animals: f-1
For a total of 6 activations.
```

Newest fact first

Salience always 0

Define rule priority

Example

animals.clp

```
(deffacts startup  ; To assert 3 facts every time the system is reset
  (animal dog)
  (animal duck)
  (animal cat)) ; even if they are retracted they will reappear after a (reset)

(defrule animal   ; prints "animal found" 3 times
  (animal ?)
  =>
  (printout t "animal found" crlf))

(defrule list-animals   ; prints the name of each animal found
(animal ?name)
=>
(printout t ?name " found" crlf))
```

```
CLIPS> (agenda)
0       animal: f-3
0       list-animals: f-3
0       animal: f-2
0       list-animals: f-2
0       animal: f-1
0       list-animals: f-1
For a total of 6 activations.
CLIPS> (set-strategy breadth)
depth
CLIPS> (agenda)
0       list-animals: f-1
0       animal: f-1
0       list-animals: f-2
0       animal: f-2
0       list-animals: f-3
0       animal: f-3
For a total of 6 activations.
```

In order of assertion

Example

animals.clp

```
(deffacts startup  ; To assert 3 facts every time the system is reset
  (animal dog)
  (animal duck)
  (animal cat)) ; even if they are retracted they will reappear after a (reset)

(defrule animal   ; prints "animal found" 3 times
  (animal ?)
  =>
  (printout t "animal found" crlf))

(defrule list-animals   ; prints the name of each animal found
(animal ?name)
=>
(printout t ?name " found" crlf))
```

```
(defrule animal   ; prints "animal found" 3 times
  (declare (salience 5))
  (animal ?)
  =>
  (printout t "animal found" crlf))
```

```
CLIPS> (agenda)
5      animal: f-1
5      animal: f-2
5      animal: f-3
0      list-animals: f-1
0      list-animals: f-2
0      list-animals: f-3
For a total of 6 activations.
```

Example

```
(deffacts startup  ; To assert facts every time the system is reset
  (animal dog)
  (animal duck)
  (animal cat)
  (warm-blooded dog)
  (warm-blooded cat)
  (warm-blooded duck)
  (lays-eggs duck)
  (lays-eggs turtle)
  (child-of dog puppy)
  (child-of cat kitten)
  (child-of turtle hatchling))


(defrule list-animals   ; prints the name of each animal found
   (animal ?name)
   =>
   (printout t ?name " found" crlf))

(defrule mammal
  (animal ?name)
  (warm-blooded ?name)
  (not (lays-eggs ?name))
  =>
  (assert (mammal ?name))
  (printout t ?name " is a mammal" crlf))

(defrule remove-mammals   ; remove the fact (animal ?) if it is a mammal
  ?fact <- (mammal ?animal)
  ?fact2 <- (animal ?animal)
  =>

  (printout t "retracting " ?fact2 crlf)
  (retract ?fact2))
```

```
(deffacts startup  ; To assert 3 facts every time the system is reset
  (animal dog)
  (animal duck)
  (animal cat)
  (warm-blooded dog)
  (warm-blooded cat)
  (warm-blooded duck)
  (lays-eggs duck)
  (lays-eggs turtle)
  (child-of dog puppy)
  (child-of cat kitten)
  (child-of turtle hatchling))

; Define a function to print a message
(deffunction print-message (?text ?name)
   (printout t ?name " " ?text crlf))

(defrule list-animals   ; prints the name of each animal found
   (animal ?name)
   =>
   (print-message "found" ?name))

(defrule mammal
  (animal ?name)
  (warm-blooded ?name)
  (not (lays-eggs ?name))
  =>
  (assert (mammal ?name)))
;  (print-message "is a mammal" ?name))

(defrule remove-mammals   ; remove the fact (animal ?) if it is a mammal
  ?fact <- (mammal ?animal)
  ?fact2 <- (animal ?animal)
  =>
  (print-message "is being retracted" ?animal)
  (retract ?fact2)
  (retract ?fact))
```

Example

```
(deffacts startup  ; To assert facts every time the system is reset
  (animal dog)
  (animal duck)
  (animal cat)
  (warm-blooded dog)
  (warm-blooded cat)
  (warm-blooded duck)
  (lays-eggs duck)
  (lays-eggs turtle)
  (child-of dog puppy)
  (child-of cat kitten)
  (child-of turtle hatchling))


(defrule list-animals   ; prints the name of each animal found
   (animal ?name)
   =>
   (printout t ?name " found" crlf))

(defrule mammal
  (animal ?name)
  (warm-blooded ?name)
  (not (lays-eggs ?name))
  =>
  (assert (mammal ?name))
  (printout t ?name " is a mammal" crlf))

(defrule remove-mammals   ; remove the fact (animal ?) if it is a mammal
  ?fact <- (mammal ?animal)
  ?fact2 <- (animal ?animal)
  =>

  (printout t "retracting " ?fact2 crlf)
  (retract ?fact2))
```

```
(deffacts startup  ; To assert 3 facts every time the system is reset
  (animal dog)
  (animal duck)
  (animal cat)
  (warm-blooded dog)
  (warm-blooded cat)
  (warm-blooded duck)
  (lays-eggs duck)
  (lays-eggs turtle)
  (child-of dog puppy)
  (child-of cat kitten)
  (child-of turtle hatchling))

; Define a function to print a message
(deffunction print-message (?text ?name)
   (printout t ?name " " ?text crlf))

(defrule list-animals   ; prints the name of each animal found
   (animal ?name)
   =>
   (print-message "found" ?name))

(defrule mammal
  (animal ?name)
  (warm-blooded ?name)
  (not (lays-eggs ?name))
  =>
  (assert (mammal ?name)))
;  (print-message "is a mammal" ?name))

(defrule remove-mammals   ; remove the fact (animal ?) if it is a mammal
  ?fact <- (mammal ?animal)
  ?fact2 <- (animal ?animal)
  =>
  (print-message "is being retracted" ?animal)
  (retract ?fact2)
  (retract ?fact))
```

Example using stages

```
; Start the process by asserting the initial stage
(deffacts initial-state
   (stage asking-symptoms))  ; We start by asking symptoms

; Rule to ask about fever
(defrule ask-symptom-fever
   (stage asking-symptoms)
   (not (asked fever))
   =>
   (printout t "Does the patient have fever? (yes/no): ")
   (bind ?response (read))
   (if (eq ?response yes) then (assert (fever)))
   (assert (asked fever))
)

; Rule to ask about cough
(defrule ask-symptom-cough
   (stage asking-symptoms)
   (not (asked cough))
   =>
   (printout t "Does the patient have cough? (yes/no): ")
   (bind ?response (read))
   (if (eq ?response yes) then (assert (cough)))
   (assert (asked cough))
)

; Rule to ask about body ache
(defrule ask-symptom-body-ache
   (stage asking-symptoms)
   (not (asked body-ache))
   =>
   (printout t "Does the patient have body ache? (yes/no): ")
   (bind ?response (read))
   (if (eq ?response yes) then (assert (body-ache)))
   (assert (asked body-ache))
)
```

```
; Rule to ask about sneezing
(defrule ask-symptom-sneezing
   (stage asking-symptoms)
   (not (asked sneezing))
   =>
   (printout t "Does the patient have sneezing? (yes/no): ")
   (bind ?response (read))
   (if (eq ?response yes) then (assert (sneezing)))
   (assert (asked sneezing))
)

; Rule to ask about mild fever
(defrule ask-symptom-mild-fever
   (stage asking-symptoms)
   (not (asked mild-fever))
   =>
   (printout t "Does the patient have mild fever? (yes/no): ")
   (bind ?response (read))
   (if (eq ?response yes) then (assert (mild-fever)))
   (assert (asked mild-fever))
)

; Only move to the "diagnosis" stage when all symptoms are asked
(defrule move-to-diagnosis
   (stage asking-symptoms)   ; Still in symptom-asking stage
   (asked fever)
   (asked cough)
   (asked body-ache)
   (asked sneezing)
   (asked mild-fever)
   =>

   (assert (stage diagnosis))          ; Move to diagnosis stage
)

; Rule to diagnose flu
(defrule flu-diagnosis
   (stage diagnosis)   ; Only diagnose when in the correct stage
   (fever)
   (cough)
   (body-ache)
   =>
   (assert (diagnosis flu))
   (printout t "Diagnosis: The patient likely has the flu." crlf))
```

Example using stages

```
; Rule to diagnose flu
(defrule flu-diagnosis
   (stage diagnosis)   ; Only diagnose when in the correct stage
   (fever)
   (cough)
   (body-ache)
   =>
   (assert (diagnosis flu))
   (printout t "Diagnosis: The patient likely has the flu." crlf))

; Rule to diagnose cold
(defrule cold-diagnosis
   (stage diagnosis)   ; Only diagnose when in the correct stage
   (sneezing)
   (mild-fever)
   =>
   (assert (diagnosis cold))
   (printout t "Diagnosis: The patient likely has a cold." crlf))

; If no diagnosis was found, print "no illness"
(defrule no-diagnosis
   (stage diagnosis)   ; Only check for diagnosis after all symptoms are asked
   (not (diagnosis ?)) ; No diagnosis exists
   =>
   (printout t "The patient does not have any diagnosed illness." crlf)
)
```

How to run the example

```
CLIPS> (clear)
CLIPS> (load "UsingStages.clp")
$*********
TRUE
CLIPS> (reset)
==> f-1     (stage asking-symptoms)
CLIPS> (run)
Does the patient have fever? (yes/no):
```

```
Does the patient have fever? (yes/no): yes
==> f-2     (fever)
==> f-3     (asked fever)
Does the patient have cough? (yes/no): no
==> f-4     (asked cough)
Does the patient have body ache? (yes/no): no
==> f-5     (asked body-ache)
Does the patient have sneezing? (yes/no): yes
==> f-6     (sneezing)
==> f-7     (asked sneezing)
Does the patient have mild fever? (yes/no): no
==> f-8     (asked mild-fever)
==> f-9     (stage diagnosis)
The patient does not have any diagnosed illness.
```

```
; Start the process by asserting the initial stage
(deffacts initial-state
   (stage asking-symptoms))  ; We start by asking symptoms

; Rule to ask about fever
(defrule ask-symptom-fever
   (stage asking-symptoms)
   (not (asked fever))
   =>
   (printout t "Does the patient have fever? (yes/no): ")
   (bind ?response (read))
   (if (eq ?response yes) then (assert (fever)))
   (assert (asked fever))
)
```

Using CLIPS with Python

- pip install clipspy

```python
import clips

env = clips.Environment()

# Define rules
env.build("""
(defrule suggest-travel
   (weather hot)
   (activity beach)
   =>
   (printout t "Suggested Destination: Maldives" crlf))
""")

# Assert user preferences
env.assert_string("(weather hot)")
env.assert_string("(activity beach)")

# Run the expert system
env.run()
```

Exercise

A bank asks you to design an expert system to help decide whether a customer's loan application should be approved, rejected or require further review.

Discuss whether this might be a suitable problem for an expert system, and say how you might start acquiring the necessary expert knowledge.

Exercise – Acquiring expert knowledge

To build the knowledge base, you can:

- Check banking regulations and lending policies
- Interview loan officers and risk analysts
- Analyze past approved and rejected loan applications
- Identify credit risk assessment criteria
- Etc..

Exercise – Acquiring expert knowledge

- Search for factors to consider (monthly income, credit score, employment stability, existing debts, age and financial history, etc)
- Search for common decision patterns:
  - Applicants with high income and stable jobs are usually approved
  - Applicants with poor credit history are often rejected
  - High loan amounts with unstable employment require additional guarantees
- Example rule: IF income is high AND credit_score is good AND debt_to_income ratio is low THEN approve loan

Exercise 2

A car repair company asks you to design an expert system to help customers identify possible causes of car problems before visiting a mechanic.

Discuss whether this might be a suitable problem for an expert system, and say how you might start acquiring the necessary expert knowledge.

Exercise – Acquiring expert knowledge

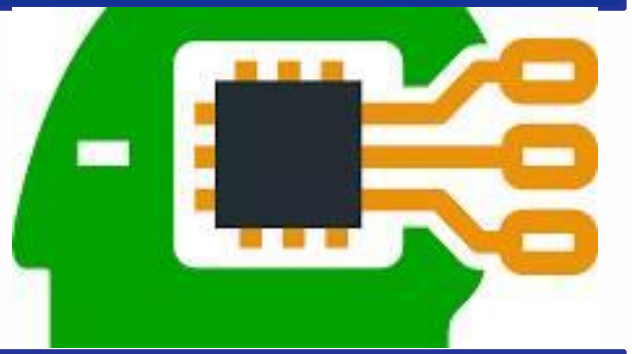
To build the knowledge base, you can:

- Interview experienced mechanics
- Study car diagnostic manuals
- Analyze common repair cases
- Examine troubleshooting flowcharts used in garages
- Identify symptom-to-fault-patterns
- Etc..

Exercise – Acquiring expert knowledge

- Search for factors to consider (does the engine start, is there unusual noise, are warning lights on, etc)
- Search for common diagnostic patterns:
  - If the engine doesn't start and lights are dim -> battery problem
  - If engine overheats -> possible cooling system failure
  - If car pulls to one side -> alignment issue
- Example rule: IF engine doesn't start AND headlights are dim THEN battery might be discharged

Expert Systems in CLIPS



VUB
ARTIFICIAL
INTELLIGENCE
RESEARCH GROUP